

JAVA PROGRAMMING LAB RECORD

WEEK 4 & 5: Classes, Objects, Inheritance & Constructor Overloading

Course: Object Oriented Programming (Java)

Course Outcome: CO1

Package: record

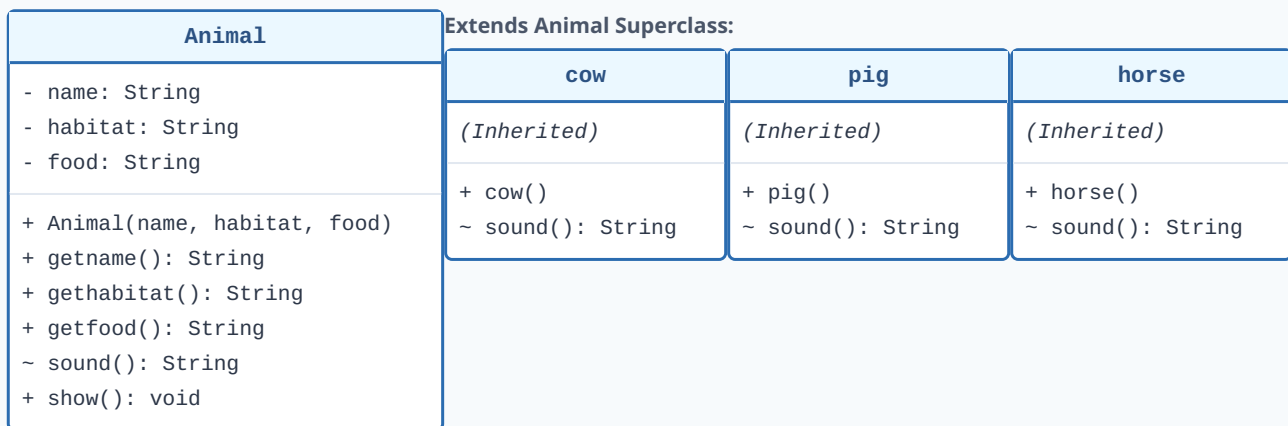
EXPERIMENT 03

Farm Animals Hierarchy (Classes and Objects)

1. PROBLEM STATEMENT

Design a "farm animals" Java application with the details of animals like cow, pig, and horse. Consider details such as where they stay (habitat), what they eat (food), and the sound they make using classes, objects, and method overriding.

2. ARCHITECTURE & UML DIAGRAM



3. EXECUTION STEP-BY-STEP PROCESS

- Base Class Definition:** Create a generic base class `Animal` containing private attributes `name`, `habitat`, and `food` with explicit getters.
- Superclass Constructor & Methods:** Define a parameterized constructor to initialize attributes, along with a default `sound()` method and a `show()` method that displays animal details.
- Subclass Inheritance:** Create specific derived classes (`cow`, `pig`, `horse`) extending `Animal`. Pass specific attributes (e.g., `habitat: "shed"`, `food: "grass"`) via `super()` calls.
- Polymorphic Overriding:** Override the `sound()` method in each derived class to return species-specific sounds ("moo", "oink", "neigh").
- Instantiation & Execution:** In the `FarmAnimals` main class, instantiate object instances for each animal type and trigger their display behaviors.

4. JAVA SOURCE CODE

```
package record;

class Animal {
    private String name;
    private String habitat;
    private String food;

    public Animal(String name, String habitat, String food) {
        this.name = name;
        this.habitat = habitat;
        this.food = food;
    }
}
```

```

    public String getname() {
        return name;
    }

    public String gethabitat() {
        return habitat;
    }

    public String getfood() {
        return food;
    }

    String sound() {
        return "animal sound";
    }

    void show() {
        System.out.println("animal is " + name);
        System.out.println("habitat is " + habitat);
        System.out.println("food is " + food);
        System.out.println("sound is " + sound());
        System.out.println("-----");
    }
}

class cow extends Animal {
    public cow() {
        super("cow", "shed", "grass");
    }

    @Override
    String sound() {
        return "moo";
    }
}

class pig extends Animal {
    public pig() {
        super("pig", "sty", "grains,vegetable");
    }

    @Override
    String sound() {
        return "oink";
    }
}

class horse extends Animal {
    public horse() {
        super("horse", "stable", "grass");
    }

    @Override
    String sound() {
        return "neigh";
    }
}

public class FarmAnimals {
    public static void main(String[] args) {
        horse farm = new horse();
        farm.sound();
        farm.show();
        pig far = new pig();
        farm.sound();
        far.show();
        cow fa = new cow();
        fa.sound();
        fa.show();
    }
}

```

```
}  
}
```

5. PROGRAM EXECUTION OUTPUT

TERMINAL CONSOLE OUTPUT

```
animal is horse  
habitat is stable  
food is grass  
sound is neigh  
-----  
animal is pig  
habitat is sty  
food is grains,vegetable  
sound is oink  
-----  
animal is cow  
habitat is shed  
food is grass  
sound is moo  
-----
```

Order Processing System (Class Diagram Implementation)

1. PROBLEM STATEMENT

Implement a Java application based on an order processing system class diagram demonstrating class relationships, inheritance, and parameter passing between objects (e.g., customer, base order, specialorder, and normalorder).

2. ARCHITECTURE & UML STRUCTURE

order	specialorder / normalorder	customer
- date: String - number: String + order(date, number) + confirm(): void + close(): void	extends order + specialorder(date, num) + normalorder(date, num) + dispatch(): void + receive(): void	- name: String - location: String + customer(n, loc) + sendorder(order): void ~ receiveorder(order): void

3. EXECUTION STEP-BY-STEP PROCESS

- Base Class Setup:** Define base class order storing general attributes like order date and tracking number with essential lifecycle methods (confirm(), close()).
- Specialized Inheritance:** Derive specialorder and normalorder subclasses extending base order functionality. Add specialized actions like dispatch() and receive().
- Association & Parameter Passing:** Implement class customer featuring methods accepting an order reference object dynamically (demonstrating dynamic binding and subtype polymorphism).
- Program Execution:** Instantiate customers and order instances in OrderProg, call dispatch/receive mechanisms, and simulate closing orders.

4. JAVA SOURCE CODE

```
package record;

class order {
    private String date;
    private String number;

    public order(String date, String number) {
        this.date = date;
        this.number = number;
    }

    public void confirm() {
        System.out.println("order " + number + " confirmed on " + date);
    }

    public void close() {
        System.out.println("order " + number + " closed");
    }
}

class specialorder extends order {
    public specialorder(String date, String number) {
        super("date", "number");
    }
}
```

```

    public void dispatch() {
        System.out.println("special order is dispatched");
    }
}

class normalorder extends order {
    public normalorder(String date, String number) {
        super("date", " number");
    }

    public void dispatch() {
        System.out.println("normal order is dispatched");
    }

    public void receive() {
        System.out.println("normal order is received by customer");
    }
}

class customer {
    private String name;
    private String location;

    public customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    public void sendorder(order order) {
        System.out.println(name + " from " + location + " sent an order");
    }

    void receiveorder(order order) {
        System.out.println(name + " received order");
        order.close();
    }
}

public class OrderProg {
    public static void main(String[] args) {
        customer c = new customer("REVANTH", "VIZIANAGARAM");
        specialorder so = new specialorder("26TH FEB", "21");
        c.sendorder(so);
        so.dispatch();
        c.receiveorder(so);

        System.out.println();

        normalorder no = new normalorder("27TH FEB", "26");
        c.receiveorder(no);
        no.dispatch();
        no.receive();
        c.receiveorder(no);
    }
}

```

5. PROGRAM EXECUTION OUTPUT

TERMINAL CONSOLE OUTPUT

```

REVANTH from VIZIANAGARAM sent an order
special order is dispatched
REVANTH received order
order number closed

REVANTH received order
order number closed
normal order is dispatched
normal order is received by customer

```

```
REVANTH received order  
order number closed
```

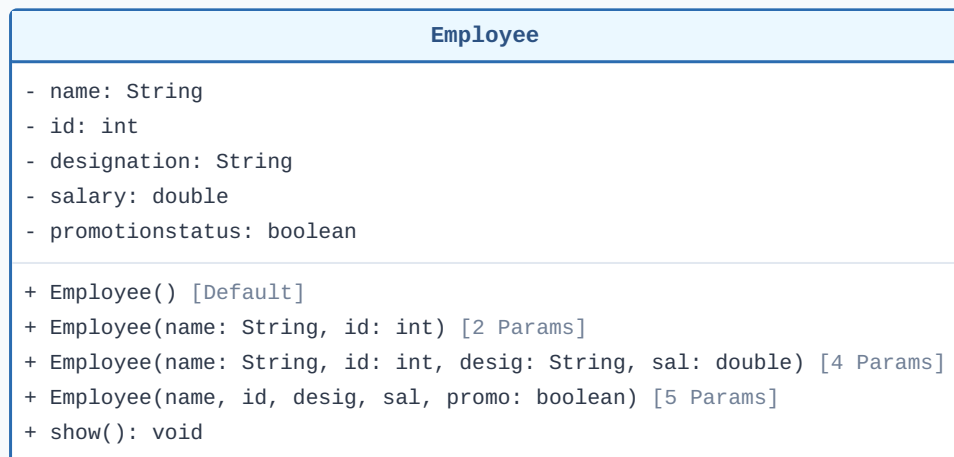
EXPERIMENT 05

Employee Data Management (Constructor Overloading)

1. PROBLEM STATEMENT

An organization maintains employee data according to employee cadre with parameters: name, id, designation, salary, and promotion status. Apply constructor overloading in Java to handle varying amounts of initialization parameters.

2. ARCHITECTURE & UML DIAGRAM



3. EXECUTION STEP-BY-STEP PROCESS

- 1. Attribute Declaration:** Create encapsulation fields inside class Employee for name, id, designation, salary, and promotionstatus.
- 2. Constructor Overloading Implementation:**
 - Default Constructor: Provides default fallbacks (e.g., name="not assigned", salary=0.0).
 - 2-Parameter Constructor: Accepts basic identifying details (name, id).
 - 4-Parameter Constructor: Sets identity plus designation and numerical salary.
 - 5-Parameter Constructor: Full state constructor accepting all parameters including promotion eligibility.
- 3. Display Method:** Implement a show() method printing full employee details to verify state initialization.
- 4. Testing Overloaded Instances:** In EmployeeData main class, instantiate e1, e2, e3, and e4 using each constructor signature respectively.

4. JAVA SOURCE CODE

```
package record;

class Employee {
    private String name;
    private int id;
    private String designation;
    private double salary;
    private boolean promotionstatus;

    // Default Constructor
    public Employee() {
        this.name = "not assigned";
        this.id = 0;
        this.designation = "trainer";
        this.salary = 0.0;
        this.promotionstatus = false;
    }
}
```

```

}

// 2 Parameter Overloaded Constructor
public Employee(String name, int id) {
    this.name = name;
    this.id = id;
    this.designation = "trainer";
    this.salary = 0.0;
    this.promotionstatus = false;
    System.out.println("--- 2 parameters passing constructor");
}

// 4 Parameter Overloaded Constructor
public Employee(String name, int id, String designation, double salary) {
    this.name = name;
    this.id = id;
    this.designation = "trainer";
    this.salary = 0.0;
    this.promotionstatus = false;
    System.out.println("4paramters passing constructor");
}

// 5 Parameter Overloaded Constructor
public Employee(String name, int id, String designation, double salary, boolean promotionstatus)
{
    this.name = name;
    this.id = id;
    this.designation = "trainer";
    this.salary = 0.0;
    this.promotionstatus = false;
    System.out.println("all details passing constructor");
}

public void show() {
    System.out.println("--- employee details ---");
    System.out.println("name : " + name);
    System.out.println("id : " + id);
    System.out.println("designation : " + designation);
    System.out.println("salary : " + salary);
    System.out.println("promotionstatus : " + promotionstatus);
}

}

public class EmployeeData {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        e1.show();

        Employee e2 = new Employee("REVANTH", 221);
        e2.show();

        Employee e3 = new Employee("REVANTH", 221, "AI Engineer", 70.000);
        e3.show();

        Employee e4 = new Employee("REVANTH", 221, "AI Engineer", 75.000, false);
        e4.show();
    }
}

```

5. PROGRAM EXECUTION OUTPUT

TERMINAL CONSOLE OUTPUT

```

--- employee details ---
name : not assigned
id : 0
designation : trainer
salary : 0.0
promotionstatus : false

```

```
--- 2 parameters passing constructor
--- employee details ---
name : REVANTH
id :221
designation : trainer
salary :0.0
promotionstatus : false
4parameters passing constructor
--- employee details ---
name : REVANTH
id :221
designation : trainer
salary :0.0
promotionstatus : false
all details passing constructor
--- employee details ---
name : REVANTH
id :221
designation : trainer
salary :0.0
promotionstatus : false
```